

UNIVERSIDADE DE AVEIRO

Departamento de Eletrónica, Telecomunicações e Informática

Redes e Sistemas Autónomos

Cooperative Lane Merge

via Comunicação V2V

Sistema autónomo de apoio à inserção cooperativa em faixa de rodagem

Pedro Melo n.º 114208

6 de junho de 2026

Repositório: [\(inserir URL\)](#)

Vídeo da demo: [\(inserir URL / Discord\)](#)

Conteúdo

1	Introdução	2
2	Objetivos	2
2.1	Problema	2
2.2	Objetivo geral	3
2.3	Objetivos específicos	3
2.4	Síntese	3
3	Arquitetura	4
3.1	Visão geral em camadas	5
3.2	Nós e topologia	6
3.3	Mapa de tópicos Zenoh	6
3.4	Modelo de papéis	7
3.5	Tipos de mensagens trocadas	7
4	Implementação	8
4.1	Visão geral	8
4.2	Fundamentos geométricos e cinemáticos	9
4.3	Mensagens e consciência situacional	11
4.4	O protocolo MergeProtocol	15
4.4.1	Lado do Merge Car (rampa)	15
4.4.2	Lado do veículo da via principal	17
4.5	A negociação de ponta a ponta	20
4.6	Orquestração, visualização e modo demo	21
5	Fluxo da Demonstração	22
5.1	Preparação	22
5.2	Timeline de uma negociação	23
5.3	Aleatoriedade e variáveis	24
5.4	Cenários	24
6	Resultados	25
7	Conclusão	25

1 Introdução

O aumento da densidade de tráfego nas entradas de autoestrada torna a inserção de veículos provenientes de rampas de acesso um dos pontos mais críticos para a fluidez e a segurança rodoviária. Tradicionalmente, esta manobra depende do julgamento individual de cada condutor, o que frequentemente origina travagens bruscas, paragens na rampa ou inserções forçadas que propagam ondas de congestionamento para a via principal.

Este projeto, desenvolvido no âmbito da unidade curricular de Redes e Sistemas Autónomos, explora como a comunicação Veículo-a-Veículo (V2V) pode substituir esse julgamento individual por uma **negociação cooperativa e distribuída**, sem qualquer entidade central de controlo. O cenário estudado é o de um veículo — o *Merge Car* (MC) — que se aproxima da via principal a partir de uma rampa de acesso e necessita de coordenar a sua inserção com os veículos que já circulam nessa via. Em vez de impor a sua entrada, o MC inicia um protocolo de coordenação de manobra em que os veículos afetados são identificados em tempo real, decidem cooperativamente um abrandamento controlado e concedem ao MC uma janela temporal segura para a inserção.

A solução assenta integralmente em mensagens normalizadas ETSI C-ITS: *Cooperative Awareness Messages* (CAM) para a consciência situacional periódica e *Manoeuvre Coordination Messages* (MCM) para a negociação da manobra. A pilha de comunicação é fornecida pelo Vanetza-NAP, o transporte de mensagens pelo middleware Zenoh, e toda a lógica dos veículos corre em containers Docker independentes, garantindo que cada veículo é um nó autónomo que só conhece a sua vizinhança através das CAMs que recebe. Uma dashboard web em tempo real permite observar a negociação e a manobra à medida que decorrem.

Este relatório descreve os objetivos do trabalho (Secção 2), a arquitetura do sistema (Secção 3), a implementação detalhada do protocolo e dos algoritmos de decisão (Secção 4), o fluxo da demonstração (Secção 5) e os resultados obtidos nos vários cenários testados (Secção 6), incluindo casos com dois veículos a inserir-se em simultâneo.

2 Objetivos

2.1 Problema

A inserção de um veículo proveniente de uma rampa de acesso na via principal é uma das manobras com maior potencial de conflito da circulação rodoviária. Sem cooperação entre veículos, o condutor que vem da rampa só dispõe da sua própria perceção visual para encontrar uma folga: quando esta não existe, vê-se obrigado a forçar a entrada, a travar bruscamente ou mesmo a parar no final da rampa à espera de espaço. Cada uma destas reações tem custos:

- **Inserção forçada** — obriga os veículos da via principal a travagens de emergência, comprometendo a segurança;
- **Paragem na rampa** — elimina a velocidade de aproximação necessária para uma inserção suave e cria risco de colisão traseira na própria rampa;
- **Travagens reativas** — propagam-se para montante sob a forma de ondas de choque,

degradando a fluidez de todo o troço.

A raiz do problema é a *falta de informação antecipada e de coordenação*: os veículos da via principal só percebem a intenção de inserção quando o veículo da rampa já está praticamente sobre o ponto de conflito, deixando pouca margem para uma resposta suave.

2.2 Objetivo geral

O objetivo central deste trabalho é demonstrar que a comunicação Veículo-a-Veículo (V2V) permite resolver este problema através de uma **negociação cooperativa, distribuída e antecipada**, sem qualquer entidade central de controlo e assente exclusivamente em mensagens normalizadas ETSI C-ITS (CAM e MCM). Pretende-se que o veículo da rampa — o *Merge Car* (MC) — anuncie a sua intenção com antecedência suficiente para que os veículos afetados se ajustem de forma coordenada e lhe abram uma janela temporal segura, substituindo a reação tardia por uma manobra planeada.

2.3 Objetivos específicos

Para concretizar o objetivo geral, definiram-se os seguintes objetivos específicos:

- O1. Consciência situacional contínua** — cada veículo difunde periodicamente a sua posição, velocidade, aceleração e dimensões via CAM, construindo uma visão local do tráfego sem qualquer configuração prévia da vizinhança.
- O2. Protocolo de negociação de manobra** — definir e implementar um protocolo distribuído sobre MCM que permita ao MC pedir autorização e aos restantes veículos responder cooperativamente.
- O3. Detecção de conflito** — determinar, em cada veículo da via principal e em tempo real, se irá efetivamente partilhar a zona de conflito com o MC, combinando a geometria da estrada com uma janela temporal de ocupação.
- O4. Abrandamento coordenado em cadeia** — quando há conflito, calcular uma velocidade-alvo viável e propagar o pedido de abrandamento aos veículos a montante, de modo a abrir a folga necessária sem travagens bruscas.
- O5. Garantia de segurança** — assegurar que a manobra nunca compromete a segurança: se a negociação não concluir a tempo, o MC recorre a um mecanismo de *stop-and-wait* no final da rampa e volta a tentar.
- O6. Suporte a múltiplos veículos a inserir-se** — estender o protocolo a cenários com dois veículos a fundir-se simultaneamente a partir da rampa.
- O7. Observabilidade** — disponibilizar uma visualização em tempo real da simulação e da troca de mensagens, para validação e demonstração do protocolo.

2.4 Síntese

A Tabela 1 resume como cada objetivo específico é concretizado na implementação, remetendo para a secção correspondente.

Objetivo	Mecanismo de concretização	Secção
O1 – Consciência situacional	Beaconing CAM a 100 ms + <code>NeighbourTable</code> com janela de 200 ms	4
O2 – Protocolo de negociação	6 subtipos de MCM (<code>MERGE_REQUEST</code> , <code>SLOW-DOWN_REQUEST/GRANT</code> , <code>MERGE_GRANT</code> , <code>MERGE_CONFIRMED</code> , <code>EXECUTION_STATUS</code>)	4
O3 – Detecção de conflito	Projeção geométrica (<code>project_t</code>) + sobreposição temporal entre a ocupação do MC e a entrada/saída do veículo na zona	4
O4 – Abrandamento em cadeia	Velocidade-alvo por busca binária (<code>_solve_target_speed</code>) e propagação de <code>SLOW-DOWN_REQUEST</code> ao veículo de trás	4
O5 – Garantia de segurança	Verificação cinemática de travagem (<code>can_brake_in_time</code>) e paragem em <code>stop_t</code> com retentativa	4
O6 – Múltiplos MC	Mesmo protocolo na rampa: o MC propaga <code>SLOW-DOWN_REQUEST</code> ao veículo atrás na rampa	4
O7 – Observabilidade	Ponte Zenoh→WebSocket (<code>bridge.py</code>) e dashboard HTML em tempo real	4

Tabela 1: Mapeamento dos objetivos específicos para os mecanismos de implementação.

3 Arquitetura

O sistema está organizado em quatro camadas com responsabilidades bem separadas, tal como ilustra a Figura 1. Esta secção fixa o vocabulário (nós, papéis, tópicos e mensagens) que a Secção 4 utiliza para detalhar a implementação.

3.1 Visão geral em camadas

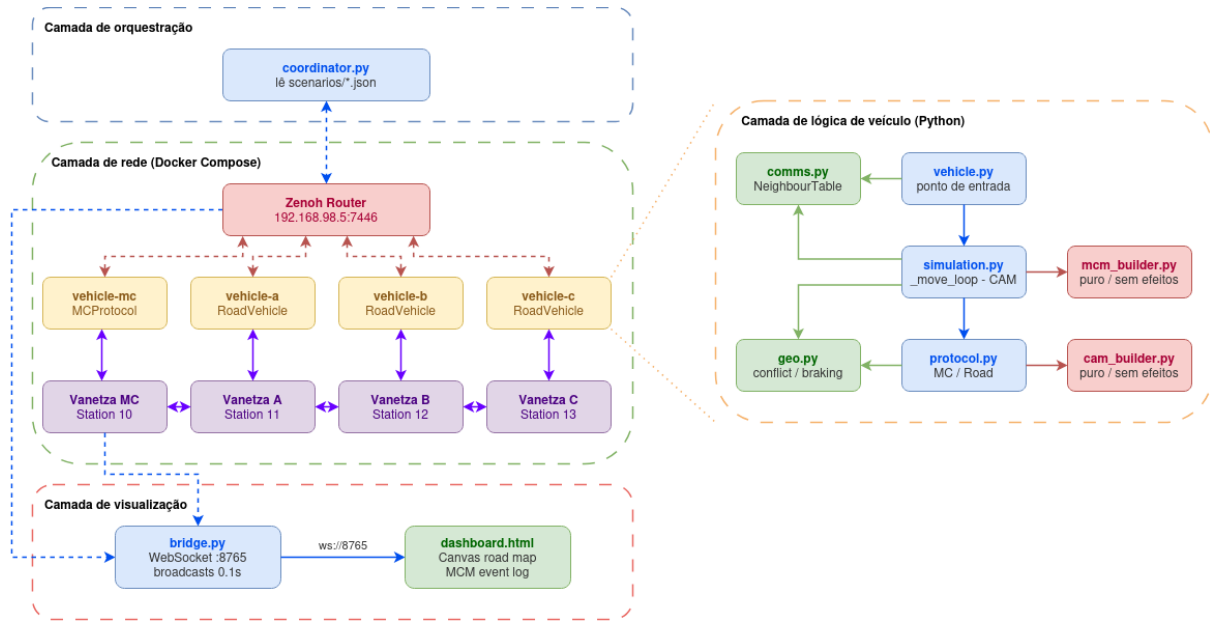


Figura 1: Arquitetura em camadas do sistema.

Camada de orquestração. Um único processo, o `coordinator.py` (GUI Tkinter), lê os ficheiros de cenário `cenarios/*.json`, publica o cenário escolhido no router Zenoh e aguarda que todos os veículos ativos reportem a conclusão. É o ponto de entrada da demonstração, mas *não* participa na negociação — apenas dá o pontapé de saída e recolhe o estado de cada veículo.

Camada de rede. Materializada por `docker-compose` sobre a rede Docker externa `vanetzalan0`. Contém o *router Zenoh* (canal do coordinator) e, por cada veículo, um par de containers: um container **Vanetza-NAP**, que implementa a pilha ETSI C-ITS (codificação/ decodificação ASN.1 e difusão pelo “ar”), e um container **Python**, que corre a lógica do veículo. É nesta camada que vivem as *stations* 10–13.

Camada de lógica de veículo (Python). É o cérebro de cada nó, organizado em módulos: `vehicle.py` (ponto de entrada), `simulation.py` (ciclo de movimento e emissão de CAM), `protocol.py` (a classe `MergeProtocol`, que concentra toda a negociação), apoiados por `geo.py` (geometria e modelo de travagem), `comms.py` (sessões Zenoh e `NeighbourTable`) e pelos construtores puros de mensagens `cam_builder.py` e `mcm_builder.py`. Esta camada é o objeto central da Secção 4.

Camada de visualização. O `bridge.py` subscreve o tráfego V2X (CAM e MCM) decodificado, mantém o estado de cada veículo e difunde, a cada 100 ms, uma trama JSON por WebSocket (porta 8765) para a `dashboard.html`, que desenha as estradas, os veículos e o registo de eventos MCM em tempo real.

Os dois planos de comunicação. A Figura 1 distingue, por cor, dois planos. O *plano V2X* (ligações entre cada veículo e o seu Vanetza e entre os Vanetza) transporta o tráfego C-ITS (CAM/MCM) e representa o meio rádio: na realidade, cada veículo difunde-o pelo broker Zenoh do seu *próprio* container Vanetza-NAP (`tcp/localhost:7447`) — não existe broker partilhado para o V2X, e a figura abstrai esses brokers como ligações diretas. O *plano de orquestração* (ligações ao bloco central “Zenoh Router”, `192.168.98.5:7446`) transporta apenas as mensagens de controlo da demonstração (cenário, *ready*, *done*). Assim, nenhum veículo comunica com outro por um canal partilhado privilegiado — toda a coordenação inter-veicular acontece exclusivamente através de mensagens C-ITS difundidas pelo respetivo Vanetza.

3.2 Nós e topologia

O cenário envolve **quatro veículos** — o Merge Car (MC) e os veículos da via principal A, B e C. Cada veículo é, na prática, um *par de containers* (Vanetza-NAP + Python) com uma identidade C-ITS fixa, resumida na Tabela 2. A estes junta-se o router Zenoh (canal de orquestração) e a cadeia de observabilidade — o `bridge.py` e a `dashboard` —, que apenas consome o tráfego decodificado para visualização, sem participar na negociação.

Veículo	Papel	Station ID	MAC	IP Vanetza
MC	Merge Car (rampa)	10	6e:06:e0:03:00:10	192.168.98.10
A	Via principal	11	6e:06:e0:03:00:11	192.168.98.11
B	Via principal	12	6e:06:e0:03:00:12	192.168.98.12
C	Via principal	13	6e:06:e0:03:00:13	192.168.98.13
Router Zenoh (coordinator)		—	—	192.168.98.5:7446

Tabela 2: Nós do sistema: identidade C-ITS e endereçamento. Cada veículo expõe ainda o broker Zenoh do seu Vanetza na porta 7447.

Cada container Python liga-se ao broker Zenoh do seu próprio Vanetza (`VANETZA_ZENOH_URL`) para o tráfego V2X e ao router central (`COORDINATOR_ZENOH_URL`) para o canal de orquestração. A geração nativa de CAM no Vanetza está **desativada** (`VANETZA_CAM_PERIODICITY=0`): toda a emissão de CAM é controlada pela camada Python, garantindo um *tick* de simulação coerente entre movimento e *beaconing*.

3.3 Mapa de tópicos Zenoh

A comunicação entre as camadas reduz-se a um pequeno conjunto de tópicos Zenoh (Tabela 3). Os tópicos `vanetza/in/*` e `vanetza/out/*` representam, respetivamente, a injeção de uma mensagem na pilha C-ITS e a sua receção após passar pelo “ar”; os tópicos `coordinator/*` constituem o canal de controlo da demonstração.

Tópico	Publica	Subscreve	Conteúdo
vanetza/in/cam	Veículo (Python)	Vanetza-NAP	CAM a difundir (dict JSON)
vanetza/out/cam	Vanetza-NAP	Veículos, <code>bridge</code>	CAM recebida do ar
vanetza/in/mcm	Veículo (Python)	Vanetza-NAP	MCM a difundir (dict JSON)
vanetza/out/mcm	Vanetza-NAP	Veículos, <code>bridge</code>	MCM recebida do ar
coordinator/scenario	<code>coordinator</code>	Veículos, <code>bridge</code>	Cenário completo (JSON)
coordinator/ready/<sid>	Veículo	<code>coordinator</code>	<i>Heartbeat</i> : veículo vivo
coordinator/done/<sid>	Veículo	<code>coordinator</code>	Veículo concluiu o cenário

Tabela 3: Mapa de tópicos Zenoh. <sid> é o *Station ID* do veículo.

3.4 Modelo de papéis

Uma decisão central da arquitetura é que **todos os veículos correm exatamente o mesmo código**. Não existe um binário “MC” e outro “veículo da estrada”: o papel *emerge em tempo de execução* do tipo de estrada onde o veículo é colocado pelo cenário. Um veículo posicionado numa estrada do tipo `ramp` com campo `merges_into` ativa o comportamento de Merge Car dentro do `MergeProtocol`; o mesmo código numa estrada `main` responde como veículo alvo da negociação. Esta unificação tem duas consequências importantes: (i) os cenários multi-MC são suportados sem código adicional — basta colocar dois veículos na rampa (ver cenários 04–06); e (ii) toda a lógica de negociação vive numa única classe, descrita em detalhe na Secção 4.

Coerentemente, **nenhum veículo tem conhecimento pré-configurado da sua vizinhança**. Quem está à frente, quem está atrás, quem entra em conflito e em que estrada cada um circula é inferido em tempo real a partir das posições GPS recebidas nas CAM e da geometria das estradas definida no cenário.

3.5 Tipos de mensagens trocadas

Apenas dois tipos de mensagem ETSI C-ITS são usados (Tabela 4). A **CAM** é o *beaconing* periódico que sustenta a consciência situacional; a **MCM** implementa o protocolo de negociação e desdobra-se em seis subtipos. Cada MCM transporta no `basicContainer` três campos que, em conjunto, identificam o subtipo — `mcmType` (tipo de mensagem), `itssRole` (papel do emissor) e `manoeuvreId` (identificador da sessão). A estrutura interna de cada mensagem, o significado destes códigos e um exemplo concreto de cada subtipo são detalhados na Secção 4.3; aqui interessa apenas o mapa geral da Tabela 4.

Subtipo	mcmType	itssRole	manoeuvreId	Direção	Propósito
MERGE_REQUEST	1	1	0–127	MC → todos	Pedir autorização; inclui zona de conflito e janela de ocupação
SLOWDOWN_REQUEST	1	3	0–127	em conflito → de trás	Propagar pedido de abrandamento na cadeia (via <code>executantID</code>)
SLOWDOWN_GRANT	2	3	128–255	atrás → à frente	Confirmar/recusar a aceitação do abrandamento
MERGE_GRANT	2	3	0–127	cada veículo → MC	Conceder a passagem ao MC, individualmente
MERGE_CONFIRMED	2	1	0–127	MC → todos	Confirmar (<i>agree</i>) ou abortar a manobra
EXECUTION_STATUS	7	1	0–127	MC → todos	MC já entrou na via; veículos retomam a velocidade

Tabela 4: CAM (*beaconing* periódico) e os seis subtipos de MCM do protocolo. O significado dos códigos de `mcmType`, `itssRole` e `manoeuvreId` consta da Tabela 6.

4 Implementação

Esta secção detalha o *cérebro* de cada nó — a camada de lógica de veículo em Python —, onde vive toda a negociação. A exposição segue uma ordem *bottom-up*, construindo o vocabulário antes de o usar: começa por uma vista geral do código e das constantes (Secção 4.1); formaliza os fundamentos geométricos e cinemáticos — a coordenada paramétrica, a geometria das estradas e o modelo de travagem (Secção 4.2); descreve as mensagens trocadas, com um exemplo concreto de cada (Secção 4.3); só então detalha o protocolo e as suas máquinas de estados, que assentam sobre tudo o anterior (Secção 4.4); recapitula a negociação de ponta a ponta num cenário concreto (Secção 4.5); e termina na orquestração e visualização (Secção 4.6).

4.1 Visão geral

Toda a lógica vive em `merge_lane/vehicle/`, repartida por módulos com responsabilidades estreitas:

- `vehicle.py` — ponto de entrada do container;
- `simulation.py` — o ciclo de movimento (`_move_loop`) e a emissão de CAM a cada *tick*;
- `protocol.py` — a classe `MergeProtocol`, que concentra *toda* a negociação (Secção 4.4);
- `geo.py` — geometria das estradas e modelo cinemático de travagem (Secção 4.2);
- `comms.py` — sessões Zenoh e a `NeighbourTable`;
- `cam_builder.py` e `mcm_builder.py` — construtores puros de mensagens (Secção 4.3).

Um pequeno conjunto de **constantes globais** (Tabela 5), definidas em `geo.py` e `protocol.py`, reaparece ao longo de toda a descrição e fixa o comportamento temporal e cinemático do sistema.

Constante	Valor	Papel
DT	0,1 s	período do <i>tick</i> de simulação (e de CAM)
ACCEL = DECEL	4,0 m/s ²	taxa de aceleração e de travagem
SAFETY_GAP	10 m	folga de segurança nas margens da zona de conflito
CONFLICT_HORIZON_S	4 s	ETA ao <i>merge point</i> a que o MC inicia a negociação
MERGE_REQUEST_RESEND_S	1 s	intervalo mínimo de reenvio do MERGE_REQUEST
RETRY_COOLDOWN_S	1 s	espera após um <i>abort</i> antes de nova tentativa

Tabela 5: Constantes globais usadas ao longo da implementação.

Uma decisão estruturante — detalhada na Secção 4.4 — é que **todos os veículos correm exatamente o mesmo código**: o papel (Merge Car ou veículo da via principal) *emerge em tempo de execução* do tipo de estrada onde o veículo é colocado, e nenhum veículo tem conhecimento pré-configurado da sua vizinhança.

4.2 Fundamentos geométricos e cinemáticos

Esta secção formaliza as primitivas de `geo.py` que sustentam todos os cálculos do protocolo: traduzem as posições GPS recebidas nas CAMs em quantidades operacionais e modelam a dinâmica de travagem. Todas as estradas são segmentos de reta definidos no cenário por **start** (início) e **end** (fim).

A coordenada paramétrica t . O conceito central de toda a localização é simples: como cada estrada é um segmento de reta, a posição de um veículo ao longo dela resume-se a uma única **coordenada paramétrica** $t \in [0, 1]$, com $t = 0$ no **start** e $t = 1$ no **end**. Toda a noção de posição relativa — quem vai à frente, quem vai atrás, onde começa e acaba a zona de conflito — reduz-se a comparar valores de t *na mesma estrada*. O comprimento físico do segmento, $L = \text{road_length}$ (calculado por *haversine*, ver adiante), converte diferenças de t em metros: uma separação Δt corresponde a $\Delta t \cdot L$ metros. É esta coordenada que reaparece em praticamente todas as fórmulas seguintes.

Distância e projeção. A distância métrica entre dois pontos GPS usa a fórmula de *haversine* (*haversine*, $R = 6\,371\,000$ m):

$$d = 2R \arcsin \sqrt{\sin^2 \frac{\Delta \varphi}{2} + \cos \varphi_1 \cos \varphi_2 \sin^2 \frac{\Delta \lambda}{2}}.$$

O valor de t de um ponto P obtém-se pela projeção escalar sobre o segmento (`project_t`):

$$t = \frac{(P - s) \cdot (e - s)}{\|e - s\|^2}, \quad s = \text{start}, e = \text{end},$$

calculada diretamente em graus (a anisotropia lat/lon é desprezável à escala local). O rumo do segmento vem de `compute_bearing` (azimute referenciado a Norte). Esta projeção é o que

permite, mais adiante, a um veículo da via principal reposicionar na sua própria estrada a zona de conflito anunciada pelo MC (Secção 4.4.2).

Pertença a uma estrada. Como na simulação os veículos se movem por interpolação exata sobre a reta, a distância entre a posição GPS e a sua projeção é ~ 0 m; `is_on_road` considera um veículo “nesta estrada” se essa distância for inferior a um limiar de 0,05 m (apenas para absorver erro de vírgula flutuante). É este teste que permite ao MC distinguir, em tempo real, quais dos vizinhos circulam na via principal (os *active peers*, Secção 4.4.1).

Vizinho de trás e movimento. `find_vehicle_behind` percorre o *snapshot* da `NeighbourTable` (Secção 4.3) e devolve o veículo com o maior $t_n < t_{\text{próprio}}$ que esteja na mesma estrada e dentro de 200 m — a base da propagação em cadeia (Secção 4.4.2). O movimento de cada veículo é um avanço incremental ao longo do segmento, $t += \text{speed} \cdot \text{DT}/L$, executado a cada *tick* pelo `_move_loop`.

Dinâmica longitudinal. A dinâmica é deliberadamente simples e determinística, com aceleração limitada e simétrica $\text{ACCEL} = \text{DECEL} = 4,0 \text{ m/s}^2$. `advance_speed` aproxima a velocidade do alvo em cada *tick*, no máximo $\pm \text{DECEL} \cdot \text{DT}$, devolvendo também a aceleração efetiva (que é depois anunciada na CAM). Sobre esta primitiva constroem-se duas funções de previsão usadas na negociação:

- `predict_motion(v, v_alvo, horizonte)` — integra `advance_speed` passo a passo e devolve a distância percorrida e a velocidade final, respeitando o limite de aceleração (não é cinemática de velocidade constante); base da velocidade-alvo e da janela de ocupação (Secções 4.4.1 e 4.4.2);
- `time_to_cover_distance(v, v_alvo, d)` — o inverso: o tempo necessário para cobrir d , ou ∞ se for impossível (e.g. parado com alvo nulo); base do ETA e dos instantes de entrada/saída na deteção de conflito.

Viabilidade de travagem. `can_brake_in_time` usa a equação de Torricelli para verificar se um veículo consegue desacelerar de v até v_{alvo} dentro da distância disponível:

$$d_{\text{trav}} = \frac{v^2 - v_{\text{alvo}}^2}{2 \cdot \text{DECEL}}, \quad \text{aceita se } d_{\text{trav}} \leq d_{\text{disp}}.$$

Este é o teste de segurança (O5) aplicado por cada veículo no fim de cadeia antes de se comprometer com um abrandamento (Secção 4.4.2) e pelo MC antes do *stop-and-wait* (Secção 4.4.1).

Dimensionamento da zona de conflito. A zona de conflito é o troço da via principal que o MC irá ocupar. As suas margens (em metros), fixadas em `_init_merge_geometry`, refletem o espaço físico que a manobra exige:

$$\text{before_m} = \frac{L_{\text{veh}}}{2} + \text{SAFETY_GAP} + \underbrace{\frac{v_{\text{main}}^2 - v_{\text{mc}}^2}{2 \cdot \text{DECEL}}}_{\text{merge_entry_margin}}, \quad \text{after_m} = \frac{L_{\text{veh}}}{2} + \text{SAFETY_GAP}.$$

A meia-carroçaria $L_{\text{veh}}/2$ e o $\text{SAFETY_GAP} = 10\text{m}$ são comuns aos dois lados; a montante acrescenta-se ainda a $\text{merge_entry_margin}$, que é precisamente a distância de travagem necessária para um veículo da via principal (a v_{main}) igualar a velocidade de entrada do MC (a v_{mc}) — nula quando $v_{\text{main}} \leq v_{\text{mc}}$. Esta assimetria reserva mais espaço atrás do *merge point*, onde o diferencial de velocidade é maior. A partir destas margens, conflict_zone_t exprime a zona em coordenadas paramétricas, centrada no *merge point* t_{merge} :

$$t_{\text{start}} = t_{\text{merge}} - \frac{\text{before_m}}{L}, \quad t_{\text{end}} = t_{\text{merge}} + \frac{\text{after_m}}{L}.$$

É esta zona que o MC codifica nos *waypoints* do MERGE_REQUEST (Secção 4.3) e que cada veículo da via principal reprojecta na sua estrada (Secção 4.4.2).

Ponto de paragem do MC. Como salvaguarda (O5), o MC nunca ultrapassa um ponto da rampa sem ter decidido o *merge*. mc_stop_t fixa essa posição paramétrica deixando a frente do veículo a um *buffer* de 8 m do fim da rampa:

$$\text{stop_t} = 1 - \frac{L_{\text{veh}} + 8}{L_{\text{ramp}}}.$$

4.3 Mensagens e consciência situacional

Esta secção descreve como as mensagens são construídas, transportadas e recebidas, como se estrutura cada tipo — com um exemplo concreto de cada — e como delas emerge a consciência situacional que alimenta todo o protocolo.

Ciclo de vida de uma mensagem. O ciclo é uniforme para CAM e MCM e atravessa as quatro camadas da Secção 3. Do lado emissor, um construtor puro devolve um dicionário: build_cam é invocado a cada *tick* pelo ciclo de simulação ($\text{simulation_move_loop}$), enquanto os build_* de mcm_builder.py são chamados pela classe MergeProtocol (Secção 4.4) quando a negociação o exige. Esse dicionário é serializado em JSON e publicado no tópico de *entrada* da pilha C-ITS:

```
vanetza_session.put("vanetza/in/cam", json.dumps(cam).encode())
```

O container Vanetza-NAP subscrive vanetza/in/* , aplica as escalas inteiras do *codec*, codifica a PDU em ASN.1/UPER e difunde-a pelo “ar”. Do lado recetor, a PDU volta a ser decodificada e republicada em vanetza/out/* , onde os *callbacks* de cada veículo (make_cam_callback , $\text{MergeProtocol.make_mcm_callback}$) e o bridge.py a consomem. O *payload* decodificado chega encapsulado em payload["fields"] — sob a chave *cam* para CAM e *payload* para MCM —, e cada *callback* começa por descartar a mensagem se o *stationID* for o seu próprio.

Beaconing e NeighbourTable (O1). A consciência situacional assenta num *beaconing* CAM síncrono com o movimento. Em cada *tick* de $\text{DT} = 0,1\text{s}$, o move_loop calcula a posição interpolada, atualiza a velocidade e constrói uma CAM com build_cam (posição, rumo, velocidade, aceleração do *tick* e comprimento), publicando-a em vanetza/in/cam . Como a geração nativa

de CAM no Vanetza está desativada (`VANETZA_CAM_PERIODICITY=0`), há exatamente uma CAM por *tick* de simulação, garantindo coerência temporal entre o que cada veículo *faz* e o que *anuncia*. Do lado da receção, `make_cam_callback` extrai do `basicVehicleContainerHighFrequency` a posição, velocidade, rumo e comprimento e invoca `NeighbourTable.update`. A `NeighbourTable` (em `comms.py`) é uma tabela *thread-safe* que guarda apenas o estado mais recente de cada vizinho, carimbado com o instante de receção. O método central é `snapshot(max_age_s=0.2)`, usado por toda a lógica de protocolo: devolve somente as entradas recebidas nos últimos 200 ms. Com CAMs a cada 100 ms, esta janela tolera exatamente *uma* CAM perdida antes de um vizinho “desaparecer” da vista, evitando decisões sobre informação obsoleta. Assim, **toda** a noção de vizinhança — quem existe, onde está, a que velocidade — é derivada exclusivamente das CAMs recebidas, sem qualquer configuração prévia.

Anatomia de uma CAM. `build_cam` (`cam_builder.py`) devolve a estrutura completa de uma CAM ETSI (TS 103 900). Por legibilidade, o excerto abaixo — de uma CAM real de um veículo a 60 km/h em velocidade constante — mostra apenas os campos que a lógica de facto consome (posição, rumo, velocidade, aceleração e comprimento); os restantes campos obrigatórios (elipses de confiança, luzes, etc.) são preenchidos com *sentinels* normalizados de “valor indisponível” (por exemplo 127, 4095 ou 800001):

```

1 {
2   "camParameters": {
3     "basicContainer": {
4       "stationType": 5,
5       "referencePosition": {
6         "latitude": 40.637,
7         "longitude": -8.651
8       }
9     },
10    "highFrequencyContainer": { "basicVehicleContainerHighFrequency": {
11      "heading": { "headingValue": 48.6993 },
12      "speed": { "speedValue": 16.6667 },
13      "longitudinalAcceleration": { "value": 0.0 },
14      "vehicleLength": { "vehicleLengthValue": 45 },
15      "driveDirection": 0
16    }
17  },
18  "generationDeltaTime": 40378
19 }

```

Os valores são fornecidos em unidades “naturais” (graus, m/s, m/s²), e é o *codec* do Vanetza que aplica as escalas inteiras do ASN.1 antes de codificar em UPER: $\times 10^7$ na latitude/longitude, $\times 100$ na velocidade e $\times 10$ na aceleração. O `vehicleLengthValue=45` já vem em unidades de 0,1 m (ou seja, 4,5 m). O `headingValue` é o azimute em graus a partir do Norte (0 = Norte, 90 = Este); o sinal da `longitudinalAcceleration` distingue travagem (negativo) de aceleração (positivo) — aqui 0,0 indica velocidade constante — e é dele que a *dashboard* infere o estado visual do veículo (Secção 4.6). O `generationDeltaTime` é um carimbo temporal em milissegundos, contador rolante em [0, 65535]. Na receção, `make_cam_callback` extrai exatamente estes campos para alimentar a `NeighbourTable`.

Construtores de MCM. `mcm_builder.py` contém **funções puras** (sem I/O): recebem o estado do veículo e devolvem o dicionário da PDU pronto a serializar; a decisão de *quando* e *a quem* enviar vive em `protocol.py` (Secção 4.4). Toda a MCM começa por um `basicContainer` comum — com o `mcmType`, o `itssRole`, o `manoeuvreId` e a posição do emissor — cuja tripla (`mcmType`, `itssRole`, `manoeuvreId`) identifica o subtipo (Tabela 4); o significado dos códigos `mcmType` e `itssRole` consta da Tabela 6. O que vem a seguir distingue um *pedido* de uma *resposta*:

Campo	Valor	Significado
<code>mcmType</code>	1	<i>request</i> — pedido de manobra
	2	<i>response</i> — resposta a um pedido
	7	<i>executionStatus</i> — estado de execução da manobra
<code>itssRole</code>	1	<i>coordinatingItss</i> — ITS-S que coordena o merge (o MC)
	3	<i>targetVehicle</i> — veículo-alvo (a quem se pede para abrandar)

Tabela 6: Significado dos códigos de `mcmType` e `itssRole` (enumerações ETSI) usados no `basicContainer`.

- **Pedidos** (`MERGE_REQUEST`, `SLOWDOWN_REQUEST`) descrevem a manobra. Levam o estado cinemático do emissor (velocidade, rumo, dimensões) e, sobretudo, uma lista `manoeuvreAdvice` em que *cada entrada endereça um destinatário* pelo seu `executantID` (o *stationID*) e lhe sugere uma velocidade — um recetor só atua sobre a entrada cujo `executantID` é o seu. O `MERGE_REQUEST` acrescenta o *Target Road Resource* (TRR), que transporta a zona de conflito: início e fim (Secção 4.2) codificados como *offsets* (Δlat , Δlon) relativos à posição do MC, o comprimento da zona e a janela de ocupação.
- **Respostas** (`MERGE_GRANT`, `SLOWDOWN_GRANT`, `MERGE_CONFIRMED`, `EXECUTION_STATUS`) são mínimas: resumem-se a um `responseContainer` com um único campo `manoeuvreResponse` (0 = aceitar/acordar, 1 = recusar/abortar).

Em concreto, o `MERGE_REQUEST` difundido pelo MC (*station* 10) tem a forma seguinte (campos abreviados); dirige-se aos três peers ativos — A (11), B (12) e C (13) —, cada um com a sua entrada em `manoeuvreAdvice`:

```

1 basicContainer: { mcmType=1, itssRole=1, stationID=10, manoeuvreId=1, concept=0,
2                   rational={ manoeuvreCooperationCost=0 } }
3 mcmContainer.vehicleManoeuvreContainer:
4   vehicleCurrentStateContainer:
5     vehicleSpeed=16.6667, vehicleHeading=48.6993, vehicleSize={ length=4.5, width=1.8,
6                       height=15 }
7   submaneuvers[0]:
8     targetRoadResourceIContainer:
9       trrType=2, laneCount=2, trrWidth=1, trrLength=862
10      waypoints[0]: { deltaLatitude=0.0003867, deltaLongitude=-0.0002968 }
11      waypoints[1]: { deltaLatitude=0.0003867, deltaLongitude=0.0007252 }
12      temporalCharacteristics: { trrOccupancyStartTime=3908, trrOccupancyEndTime=4580 }
13 manoeuvreAdvice:
14   { executantID=11, advisedSpeed=27.77 }
```

```

14 { executantID=12, advisedSpeed=27.77 }
15 { executantID=13, advisedSpeed=27.77 }

```

Os **waypoints** do TRR são *offsets* (Δlat , Δlon) relativos à posição do MC: somando-os à posição do emissor, qualquer recetor reconstrói o início e o fim da zona de conflito em coordenadas absolutas (a **trrLength**= 862 é o comprimento da zona em unidades de 0,1 m, ou seja 86,2 m). A **temporalCharateristics** carrega a **janela de ocupação** [**tRROccupancyStartTime**, **tRROccupancyEndTime**] = [3908, 4580] ms a contar de agora — o intervalo em que o MC estará dentro da zona. E cada entrada de **manoeuvreAdvice** endereça um destinatário pelo **executantID** (o seu *stationID*) com uma velocidade sugerida (aqui a de cruzeiro, 27,77 m/s $\approx$ 100 km/h); um recetor só atua sobre a entrada cujo **executantID** é o seu.

O **SLOWDOWN_REQUEST** tem a mesma forma de um pedido, mas com **itssRole**=3, um único **manoeuvreAdvice** dirigido ao veículo imediatamente atrás e sem TRR (a zona já é conhecida da cadeia); a sua janela **tRROccupancy** é um *placeholder* (1000/8000 ms). Aqui é B (12) a pedir a C (13) que abrande para $\approx$ 15,41 m/s:

```

1 basicContainer: { mcmType=1, itssRole=3, stationID=12, manoeuvreId=1, concept=0,
2                  rational={ manoeuvreCooperationCost=0 } }
3 mcmContainer.vehicleManoeuvreContainer:
4   vehicleCurrentStateContainer:
5     vehicleSpeed=27.7778, vehicleHeading=89.9974, vehicleSize={ length=4.5, width=1.8,
6                           height=15 }
7   submaneuvers[0]:
8     temporalCharateristics: { tRROccupancyStartTime=1000, tRROccupancyEndTime=8000 }
9     manoeuvreAdvice[0]: { executantID=13, advisedSpeed=15.4077 }

```

As quatro *respostas* (**MERGE_GRANT**, **SLOWDOWN_GRANT**, **MERGE_CONFIRMED** e **EXECUTION_STATUS**) partilham o mesmo corpo mínimo — um **responseContainer** com um único campo **manoeuvreResponse** (0 = aceitar/acordar, 1 = recusar/abortar) — e distinguem-se apenas pela tripla do **basicContainer**. Por exemplo, o **MERGE_GRANT**(aceitar) de B (*station* 12) para o MC:

```

1 basicContainer: { mcmType=2, itssRole=3, stationID=12, manoeuvreId=1 }
2 responseContainer: { manoeuvreResponse: 0 }

```

A Tabela 7 resume como a tripla identifica cada uma das quatro respostas. O ponto subtil é o **manoeuvreId**: **MERGE_GRANT** e **SLOWDOWN_GRANT** partilham **mcmType**=2 e **itssRole**=3 e só se distinguem pelo seu intervalo (1 byte) — **0–127** para o primeiro, **128–255** para o segundo. No exemplo, o **MERGE_GRANT** ecoa o identificador da sessão (1), enquanto um **SLOWDOWN_GRANT** da mesma negociação usou 231.

Resposta	mcmType	itssRole	manoeuvreId	manoeuvreResponse
MERGE__GRANT	2	3	0–127	0 concede / 1 recusa a passagem ao MC
SLOWDOWN__GRANT	2	3	128–255	0 trava a tempo / 1 não consegue
MERGE__CONFIRMED	2	1	0–127	0 acordo (aplicar) / 1 abortar
EXECUTION__STATUS	7	1	0–127	0 fixo: MC já entrou, retomar

Tabela 7: As quatro respostas MCM: a tripla do `basicContainer` identifica cada uma; o corpo é sempre um `responseContainer` com `manoeuvreResponse`.

4.4 O protocolo MergeProtocol

Toda a negociação está numa única classe, `MergeProtocol`, partilhada por todos os veículos (O2, O6). O papel *emerge* em tempo de execução: `set_current_road` liga `current_is_ramp` quando a estrada é do tipo `ramp` com `merges_into`, e é esse *flag* (`_is_ramp()`) que encaminha cada veículo para o comportamento de Merge Car (Secção 4.4.1) ou de veículo da via principal (Secção 4.4.2). O estado é protegido por um `threading.Lock`, pois o *tick* de movimento (uma *thread*) e os *callbacks* MCM (outra) acedem-no concorrentemente. A interface pública resume-se a `tick(...)`, chamado a cada *tick* e que devolve `{advance, target_speed}` ao ciclo de movimento, e `make_mcm_callback()`, que despacha cada MCM recebida pela tripla (`mcmType`, `itssRole`, `manoeuvreId`) (cf. Tabela 4). As duas subsecções seguintes descrevem os dois comportamentos que esta classe encerra.

4.4.1 Lado do Merge Car (rampa)

A Figura 2 resume os estados do Merge Car; descrevem-se a seguir. O comportamento concentra-se em `_tick_ramp`, executado a cada *tick*.

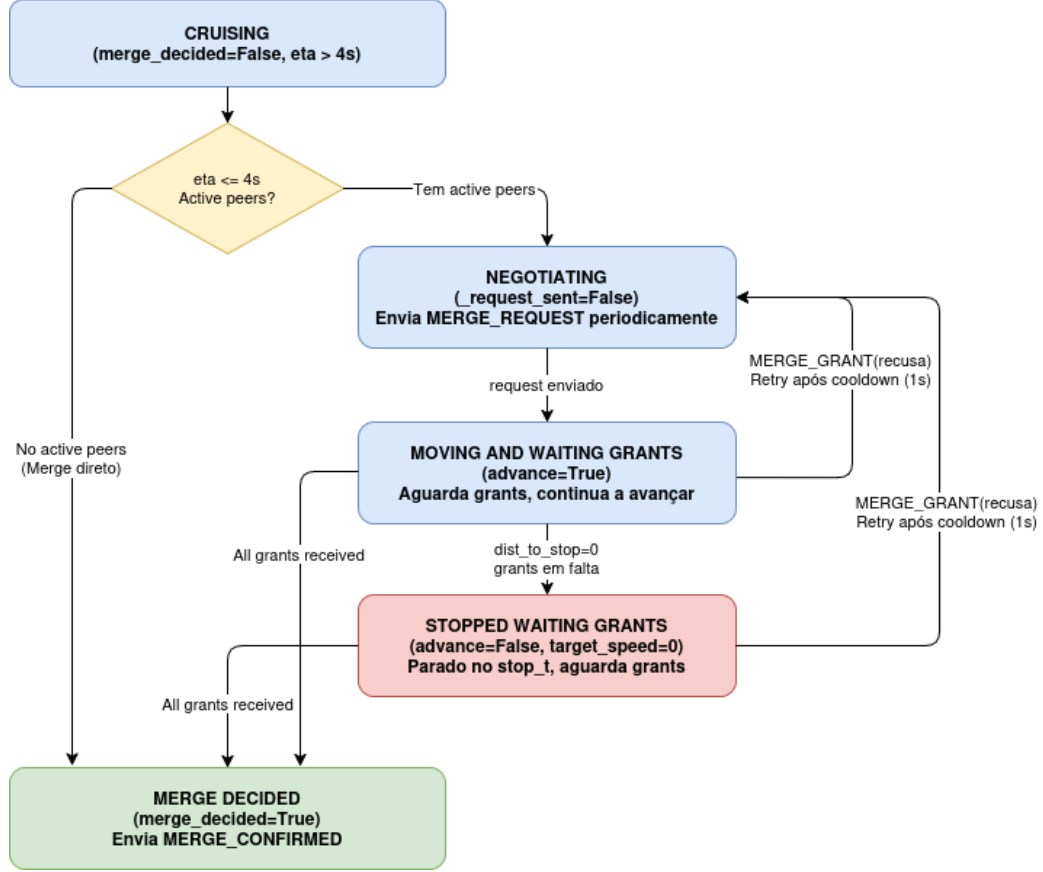


Figura 2: Máquina de estados do Merge Car.

Enquanto a velocidade é positiva, o MC estima o tempo até ao *merge point* com base na fração restante da rampa, recorrendo ao modelo cinemático (Secção 4.2):

$$\text{eta}_s = \text{time_to_cover_distance}(v, v_{\text{rampa}}, (1 - t) L_{\text{rampa}}).$$

Em **CRUISING**, enquanto $\text{eta}_s > \text{CONFLICT_HORIZON_S} = 4\text{s}$, o MC limita-se a circular. Quando o ETA desce ao horizonte de 4s, avalia os *active peers* — os vizinhos que *is_on_road* (Secção 4.2) coloca na via principal:

- **sem peers ativos** → não há com quem negociar: o MC salta direto para *_on_all_granted* (*merge* direto);
- **com peers** → entra em **NEGOTIATING** e chama *_maybe_send_merge_request*.

Construção do *merge_request*. O MC anuncia dois elementos. Primeiro, a *zona de conflito* — o troço que vai ocupar (Secção 4.2), que viaja nos *waypoints* do TRR como *offsets* relativos à sua posição. Segundo, a *janela de ocupação* dessa zona: estima a velocidade de entrada na via, $\text{entry_speed} = \text{predict_motion}(v, v_{\text{rampa}}, \text{eta}_s)$, e depois quanto tempo a ocupará, $\text{occupancy} = \text{time_to_cover_distance}(\text{entry_speed}, v_{\text{main}}, \text{after}_m)$ (ambas do modelo cinemático, Secção 4.2); a janela $[\text{eta}_s, \text{eta}_s + \text{occupancy}]$ (em ms) viaja nos campos *tRROccupancyStartTime/EndTime*. Cada envio incrementa *manoeuvre_id* de forma cíclica em $[0, 127]$ $((\text{id}+1)\%128)$ e é limitado a um por *MERGE_REQUEST_RESEND_S*=1s.

Recolha de *grants* e decisão. Cada `MERGE_GRANT(aceitar)` recebido acrescenta o emissor ao conjunto `grants_received` (estado **MOVING AND WAITING GRANTS** — o MC continua a avançar enquanto espera). Quando `active_peers  $\subseteq$  grants_received`, `_on_all_granted` marca `merge_decided` e difunde `MERGE_CONFIRMED(agree)`, transitando para **MERGE DECIDED**. Se em vez disso chegar um `MERGE_GRANT(recusar)` (`on_merge_grant` com `success=False`), o MC limpa os *grants*, difunde `MERGE_CONFIRMED(abort)` e agenda nova tentativa após `RETRY_COOLDOWN_S=1 s`.

Salvaguarda *stop-and-wait* e cadeia na rampa. Em paralelo, o MC monitoriza a distância ao `stop_t` (Secção 4.2). Se `can_brake_in_time` indicar que está prestes a alcançá-lo sem ter decidido, passa o alvo a `target_speed=0` — o que o faz travar progressivamente (à taxa `DECEL`) — e, ao imobilizar-se, fica **STOPPED WAITING GRANTS** (`advance=False`), continuando a renegociar até obter todos os *grants* (O5). Nesse estado, e relevante para cenários multi-MC (O6), `_maybe_send_ramp_slowdown` procura um veículo atrás *na própria rampa* (`find_vehicle_behind`) e propaga-lhe um `SLOWDOWN_REQUEST` — a mesma cadeia de abrandamento da via principal, agora aplicada na rampa.

4.4.2 Lado do veículo da via principal

A Figura 3 resume os estados de um veículo da via principal; descrevem-se a seguir.

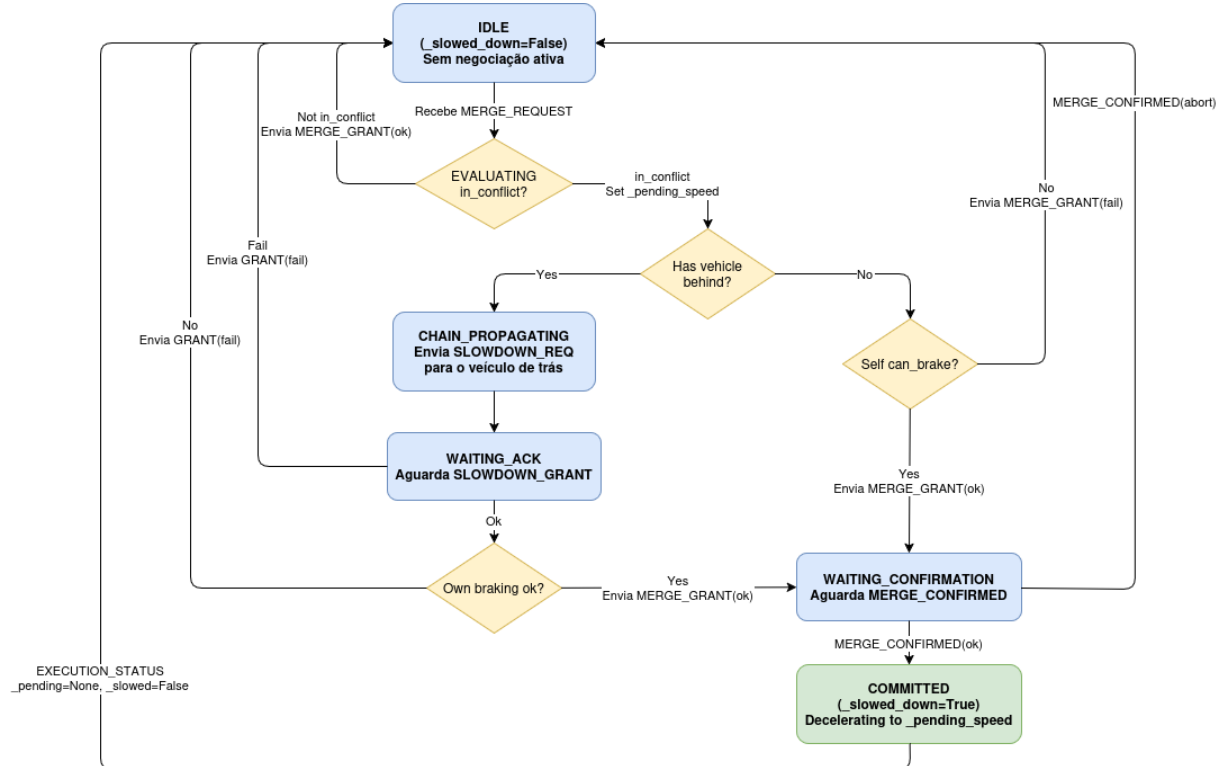


Figura 3: Máquina de estados do veículo na via principal.

Um veículo da via principal está normalmente em **IDLE**. Ao receber um `MERGE_REQUEST`, entra em `_on_merge_request` e *avalia o conflito*.

Deteção de conflito (O3). O veículo reconstrói a zona a partir dos *waypoints* do TRR (somando os *offsets* à posição do MC) e projeta-a na sua própria estrada com `project_t` (Secção 4.2), obtendo `cz_t_start` e `cz_t_end`. Calcula então as distâncias até entrar e sair da zona (descontando/somando a meia-carroçaria), converte-as nos instantes `t_enter` e `t_exit` a velocidade constante, e declara conflito se essa janela se **sobrepe**õe à janela de ocupação anunciada pelo MC, `[mc_eta_start, mc_eta_end]` (Algoritmo 1).

Algorithm 1: Deteção de conflito (`_on_merge_request`)

Input: posição t , velocidade v , zona `[cz_t_start, cz_t_end]`, janela do MC `[etas, etae]`, comprimento L_{veh} , L

Output: `in_conflict` $\in \{V, F\}$

```

1  $d_{enter} \leftarrow \max(0, (cz\_t\_start - t) L - L_{veh}/2);$ 
2  $d_{exit} \leftarrow \max(0, (cz\_t\_end - t) L + L_{veh}/2);$ 
3  $t\_enter \leftarrow \text{time\_to\_cover\_distance}(v, v, d_{enter});$ 
4  $t\_exit \leftarrow \text{time\_to\_cover\_distance}(v, v, d_{exit});$ 
5 return  $(t\_enter < eta_e) \wedge (t\_exit > eta_s);$ 

```

Sem conflito. O veículo responde de imediato com `MERGE_GRANT`(aceitar) ao MC e regressa a **IDLE** — não precisa de abrandar.

Com conflito: velocidade-alvo (O4). Detetado o conflito, o veículo tem de abrandar o suficiente para não chegar à zona enquanto o MC a ocupa, mas sem travar mais do que o necessário. `_solve_target_speed` resolve esse compromisso por busca binária (24 iterações): procura a *maior* velocidade-alvo v^* tal que, abrandando para ela, o veículo percorre *menos* do que a distância d que o separa do início da zona durante o intervalo `eta` do MC — ou seja, no instante em que o MC ocupa a zona, este veículo ainda não lá chegou. A previsão a cada passo usa o modelo cinemático real (`predict_motion`, com aceleração limitada, Secção 4.2), pelo que o v^* encontrado é mesmo atingível; maximizá-lo minimiza o abrandamento imposto e preserva a fluidez (Algoritmo 2). A velocidade-alvo fica guardada em `_pending_speed`.

Algorithm 2: Velocidade-alvo (`_solve_target_speed`)

Input: distância à zona d , ETA do MC `eta`, velocidade atual v , limite v_{max}

Output: velocidade-alvo

```

1 if  $eta \leq 0$  ou  $d \leq 0$  then return 0;
2  $lo \leftarrow 0;$   $hi \leftarrow \max(v, v_{max});$ 
3 for 24 vezes do
4    $mid \leftarrow (lo + hi)/2;$ 
5    $(d_{pred}, \_) \leftarrow \text{predict\_motion}(v, mid, eta);$ 
6   if  $d_{pred} > d$  then  $hi \leftarrow mid;$ 
7   else  $lo \leftarrow mid;$ 
8 return  $hi;$ 

```

Propagação em cadeia (O4). Com a velocidade-alvo calculada, o veículo procura um veículo imediatamente atrás (`find_vehicle_behind`, Secção 4.2):

- **há veículo atrás** $\rightarrow$ propaga a cadeia (**CHAIN_PROPAGATING**): envia-lhe `SLOWDOWN_REQUEST` com a sua sugestão e fica em **WAITING_ACK** à espera do `SLOWDOWN_GRANT`;
- **não há veículo atrás** (fim da cadeia) $\rightarrow$ verifica a própria viabilidade com `can_brake_in_time` (Secção 4.2): se consegue travar, compromete-se (`_slowed_down = True`) e envia `MERGE_GRANT`(aceitar); senão, `MERGE_GRANT`(recusar).

`_on_slowdown_request` replica esta lógica num veículo intermédio: adota a velocidade sugerida (ou $0,7v$ por omissão), e ou reenvia `SLOWDOWN_REQUEST` ao seguinte, ou — se for o fim da cadeia — testa a sua travagem e responde com `SLOWDOWN_GRANT`. Quando o `SLOWDOWN_GRANT` regressa pela cadeia, `_on_slowdown_grant` confirma que a *própria* travagem continua viável e propaga-o para a frente; o veículo no topo de cada cadeia (sem remetente à frente) traduz esse grant final no `MERGE_GRANT` que envia ao MC. Assim, um veículo em conflito só concede a passagem ao MC depois de garantir que toda a cadeia atrás de si consegue abrandar em segurança (Algoritmo 3).

Algorithm 3: Propagação em cadeia do abrandamento

Input: velocidade-alvo v_{alvo} , remetente à frente (ou MC)

```

1 _pending_speed  $\leftarrow v_{alvo}$ ;
2 b  $\leftarrow$  find_vehicle_behind(...);
3 if b  $\neq \emptyset$  then
4   enviar SLOWDOWN_REQUEST(b, v_alvo);
5   aguardar SLOWDOWN_GRANT de b ;                                // _on_slowdown_grant
6 else
7    $(ok, d_{trav}) \leftarrow can\_brake\_in\_time(v, v_{alvo}, d_{disp})$ ;
8 if cadeia atrás aceitou e própria travagem ok then
9   _slowed_down  $\leftarrow$  Verdadeiro;
10  responder GRANT(aceitar) ao remetente à frente ou MERGE_GRANT ao MC;
11 else
12  _pending_speed  $\leftarrow \emptyset$ ;
13  responder GRANT(recusar) / MERGE_GRANT(recusar);

```

Execução e retoma. O `_pending_speed` é apenas *pendente*: só é efetivamente aplicado (passa a **COMMITTED**) quando chega `MERGE_CONFIRMED`(agree) em `_on_merge_confirmed` — a partir daí, `tick()` devolve essa velocidade ao ciclo de movimento e o veículo desacelera. Um `MERGE_CONFIRMED`(abort) cancela o abrandamento e devolve-o a **IDLE**. Por fim, ao receber `EXECUTION_STATUS` (`_on_execution_status`, o MC já entrou na via), limpa `_slowed_down`/`_pending_speed` e retoma a velocidade-limite da estrada.

4.5 A negociação de ponta a ponta

Reunidas as peças — a coordenada t e o modelo cinemático (Secção 4.2), as mensagens e os seus campos (Secção 4.3) e as máquinas de estados dos dois lados (Secção 4.4) —, a Figura 4 dá a vista de conjunto do protocolo num cenário concreto: o MC funde-se na via onde circulam A, B e C (A à frente, C atrás), estando apenas **B em conflito**. A sequência decorre como se segue (os subtipos de MCM constam da Tabela 4):

1. **Pedido.** Ao atingir o horizonte de 4s, o MC difunde `MERGE_REQUEST` a todos (A, B, C), com a zona de conflito e a janela de ocupação.
2. **Grants diretos.** A e C avaliam o conflito e, não se sobrepondo à janela do MC, concedem `MERGE_GRANT` de imediato.
3. **Conflito e cadeia.** B deteta sobreposição, calcula a velocidade-alvo e, tendo C atrás de si, envia-lhe `SLOWDOWN_REQUEST` em vez de responder já ao MC.
4. **Retorno da cadeia.** C (fim de cadeia) confirma que trava a tempo e devolve `SLOWDOWN_GRANT` a B; B valida a própria travagem e só então envia o seu `MERGE_GRANT` ao MC.
5. **Confirmação.** Com `active_peers = {A, B, C} ⊆ grants_received`, o MC difunde `MERGE_CONFIRMED(agree)`. B (e a sua cadeia) aplica então o abrandamento; o MC executa a inserção.
6. **Retoma.** Já na via principal, o MC emite `EXECUTION_STATUS` e todos os veículos retomam a velocidade normal.

Ramo alternativo (ABORT). Se C não conseguir travar a tempo, devolve `SLOWDOWN_GRANT(recusar)`; B propaga a recusa como `MERGE_GRANT(recusar)` ao MC. O MC reage com `MERGE_CONFIRMED(abort)`, repõe o estado e volta a tentar após o *cooldown* de 1 s — ou, se entretanto alcançar o `stop_t`, recorre ao *stop-and-wait* (Secção 4.4.1).

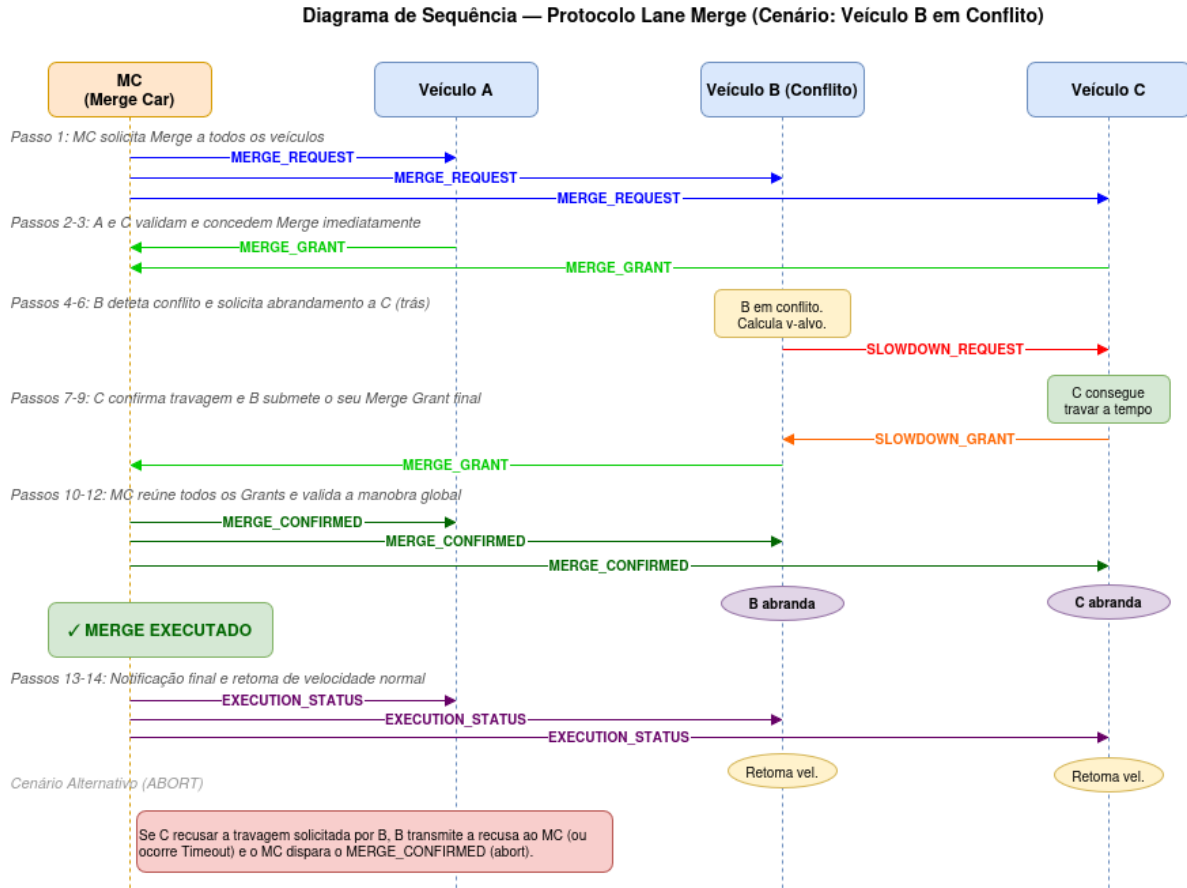


Figura 4: Diagrama de sequência do protocolo (cenário com o veículo B em conflito).

4.6 Orquestração, visualização e modo demo

Em torno da lógica de veículo existem componentes de suporte que não participam na negociação.

Coordenação (coordinator.py). Uma GUI Tkinter lê os cenários de `scenarios/*.json`, e ao selecionar um publica-o em `coordinator/scenario` no router Zenoh. Cada container de veículo subscreve esse tópico e, se o seu `station_id` constar de `active_vehicles`, corre o cenário, emitindo `heartbeats` em `coordinator/ready/<sid>` e, no fim, `coordinator/done/<sid>`. O coordenador aguarda o `done` de todos antes de avançar. Expõe ainda dois interruptores: **Demo Mode** (ativa o `DemoMergeProtocol`, descrito abaixo) e **Draw Exag** (fator de exagero visual enviado à `dashboard`). O `run.sh` encadeia todo o arranque: `docker-compose up, bridge`, servidor HTTP, a GUI e o browser.

Visualização (bridge.py + dashboard.html) (O7). O `bridge.py` é o único componente que fala simultaneamente Zenoh e WebSocket. Subscreve `vanetza/out/{cam,mcm}` e `coordinator/scenario`, mantém o estado de cada veículo e difunde uma trama JSON a cada 100 ms pela porta 8765 para a `dashboard.html` (*canvas* HTML, sem mapas). A função `_classify_mcm` traduz cada MCM num rótulo legível (e.g. `MERGE_REQUEST`, `SLOWDOWN_GRANT`), e o estado visual de cada veículo (`NORMAL`, `SLOWING`, `SPEEDING`, `MERGING`) é inferido do sinal da aceleração anunciada na CAM e das transições `MERGE_CONFIRMED/EXECUTION_STATUS`. O *merge point* é recalculado

geometricamente por `_compute_merge_point` (interseção das retas `main` e `ramp`), e a zona de conflito é reconstruída a partir dos *waypoints* do `MERGE_REQUEST`.

Modo Demo (`DemoMergeProtocol`). Para tornar a negociação observável passo a passo, `protocol_demo.py` define `DemoMergeProtocol`, uma subclasse de `MergeProtocol`. A ideia é introduzir pausas *apenas* nos pontos de decisão relevantes: ao receber uma MCM, o *callback* determina se este veículo a vai realmente processar (`will_process`) e, nesse caso, dorme `demo_pause_s` antes de delegar no *callback* do progenitor. Os veículos da via principal congelam o movimento (`_demo_paused`) ao verem o `MERGE_REQUEST` e retomam-no ao `MERGE_CONFIRMED`. Para acomodar estas pausas, o reenvio do `MERGE_REQUEST` é limitado a `_demo_resend_s = 1 + 4 · demo_pause_s`, em vez do 1 s do modo normal.

5 Fluxo da Demonstração

Esta secção descreve como a demonstração é executada e observada: a preparação do ambiente (Secção 5.1), a *timeline* narrada de uma negociação concreta (Secção 5.2), as fontes de aleatoriedade e as variáveis controláveis (Secção 5.3) e o conjunto de seis cenários preparados, com o comportamento observado em cada um (Secção 5.4).

5.1 Preparação

Todo o arranque é encadeado pelo `run.sh`, que executa a sequência: parar processos e *containers* antigos; `docker-compose up -d --build` (que recria as imagens Python e levanta o router Zenoh e, por cada veículo, o par Vanetza-NAP + Python); aguardar que o router Zenoh (127.0.0.1:7446) e o *broker* Vanetza do MC (192.168.98.10:7447) estejam disponíveis; arrancar a `bridge.py` e um servidor HTTP na porta 8000; aguardar o *WebSocket* da *bridge* (8765); abrir a `dashboard.html` no *browser*; e, por fim, arrancar a GUI `coordinator.py`.

Com o ambiente no ar, a demonstração conduz-se a partir da GUI do *coordinator* (Figura 5). O painel **Scenarios** lista os cenários de `scenarios/*.json`; o operador seleciona um e carrega em **Run Selected** (ou **Run All** para os correr em cadeia). A caixa **Demo Mode** ativa o `DemoMergeProtocol` — que introduz pausas nos pontos de decisão (Secção 4.6) — com a duração indicada em **Pausa (s)**; o campo **Draw Exag** controla o exagero visual enviado à *dashboard*. Ao iniciar, o *coordinator* publica o cenário em `coordinator/scenario`, e o painel **Vehicle Status** acompanha o estado de cada veículo (vivo / concluído) através dos *heartbeats* `coordinator/ready/<sid>` e `coordinator/done/<sid>`; o painel **Log** regista o desenrolar. A negociação propriamente dita observa-se na `dashboard.html` (`localhost:8000/dashboard.html`), que desenha as estradas, o *merge point*, a zona de conflito e os veículos coloridos pelo seu estado, mais um registo dos eventos MCM em tempo real (Secção 4.6).

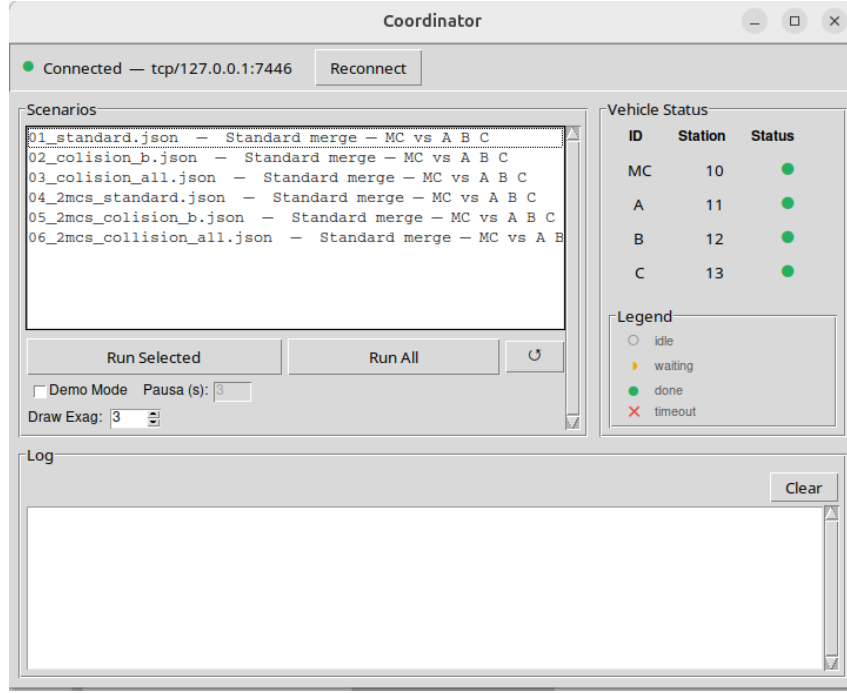


Figura 5: GUI do *coordinator*: seleção de cenário, Demo Mode e estado dos veículos.

5.2 Timeline de uma negociação

Toma-se como fio condutor o **cenário 01** (Secção 5.4), em que apenas **B entra em conflito** e *consegue* abrandar a tempo, abrindo a folga para o MC se inserir **entre A e B**. É a mesma negociação do diagrama de sequência da Figura 4, agora vista como linha temporal:

- t1. Ponto de partida.** O MC arranca na rampa e acelera (limite 60 km/h); A, B e C circulam na via principal a ≈ 100 km/h, com A à frente, B no meio e C atrás. Todos difundem CAM a cada $DT = 0,1$ s, povoando mutuamente as *NeighbourTable* (Secção 4.3).
- t2. Disparo da negociação.** À medida que o MC se aproxima, o seu ETA ao *merge point* (Secção 4.4.1) desce até $CONFLICT_HORIZON_S = 4$ s. Nesse instante o MC identifica os *active peers* (A, B, C, todos *is_on_road* na via principal) e difunde *MERGE_REQUEST* com a zona de conflito (nos *waypoints* do TRR) e a janela de ocupação.
- t3. Grants diretos.** A e C avaliam o conflito (Algoritmo 1): as suas janelas de ocupação não se sobrepõem à do MC, pelo que concedem *MERGE_GRANT*(aceitar) de imediato.
- t4. Conflito e cadeia.** B deteta sobreposição, resolve a velocidade-alvo por busca binária (Algoritmo 2) e, tendo C atrás de si, propaga-lhe *SLOWDOWN_REQUEST* em vez de responder já ao MC.
- t5. Retorno da cadeia.** C (fim de cadeia) confirma com *can_brake_in_time* que trava a tempo e devolve *SLOWDOWN_GRANT*; B revalida a própria travagem e só então envia o seu *MERGE_GRANT* ao MC.
- t6. Confirmação.** Com $active_peers = \{A, B, C\} \subseteq grants_received$, o MC difunde *MERGE_CONFIRMED*(agree); B e C aplicam o abrandamento (passam a *SLOWING* na *dashboard*) e o MC executa a inserção entre A e B.
- t7. Retoma e fim.** Já na via principal, o MC emite *EXECUTION_STATUS*; todos retomam a

velocidade-limite. O veículo reporta `coordinator/done`; quando todos os ativos concluem, o *coordinator* dá o cenário por terminado.

Esta corrida é *limpa* — uma única ronda de negociação. O registo de MCM correspondente, tal como aparece na *dashboard/log*, resume-se a oito mensagens:

```

1 [MCM] MERGE_REQUEST      MC -> all  (manoeuvre_id=1)
2 [MCM] MERGE_GRANT(OK)    A  -> MC   (manoeuvre_id=1)
3 [MCM] MERGE_GRANT(OK)    C  -> MC   (manoeuvre_id=1)
4 [MCM] SLOWDOWN_REQUEST   B  -> C    (manoeuvre_id=1)
5 [MCM] SLOWDOWN_GRANT(OK) C  -> B    (manoeuvre_id=211)
6 [MCM] MERGE_GRANT(OK)    B  -> MC   (manoeuvre_id=1)
7 [MCM] MERGE_CONFIRMED(OK) MC -> all  (manoeuvre_id=1)
8 [MCM] EXECUTION_STATUS(OK) MC -> all  (manoeuvre_id=1)

```

O `manoeuvre_id=211` do `SLOWDOWN_GRANT` (no intervalo 128–255) ilustra já a aleatoriedade discutida em seguida.

5.3 Aleatoriedade e variáveis

Fontes de aleatoriedade. Duas. Primeira, o `manoeuvre_id` de cada `SLOWDOWN_GRANT` é sorteado em `[128, 255]` (`random.randint(128, 255)`) — é o que distingue um `SLOWDOWN_GRANT` de um `MERGE_GRANT` (Tabela 7) e por isso muda a cada corrida (no exemplo acima, 211). Segunda, o *jitter* de rede e a ordem de chegada das CAM/MCM difundidas pelo “ar” (Zenoh): a ordem relativa dos *grants* de A e C, por exemplo, pode variar entre corridas. Nenhuma destas fontes altera o *desfecho* — a decisão depende apenas de `active_peers ⊆ grants_received` —, mas torna cada execução visivelmente distinta.

Variáveis controláveis. O comportamento é moldado por: o **cenário** escolhido (sobretudo as posições iniciais dos veículos, que determinam quem entra em conflito); o **Demo Mode** e a respetiva **pausa**, que abranda a negociação para observação (com o reenvio do `MERGE_REQUEST` alargado para `_demo_resend_s = 1 + 4 · pausa`, Secção 4.6); o **Draw Exag** (apenas visual); e, no JSON de cada cenário, os `speed_limit_kmh` das estradas e os `length_m` dos veículos.

5.4 Cenários

Prepararam-se seis cenários, todos com os mesmos quatro veículos (MC, A, B, C), a mesma rampa (60 km/h) e a mesma via principal (100 km/h); o que os distingue são as **posições iniciais** e o facto de, nos três últimos, A arrancar também na *rampa* (segundo Merge Car). Os nomes dos ficheiros referem-se ao conflito que ocorreria *sem* protocolo — não ao desfecho, que é sempre uma inserção segura. A Tabela 8 resume o comportamento observado em cada um.

#	Cenário	Configuração	Comportamento observado
01	standard	A, B, C na via principal	B em conflito; cadeia B→C; B e C abrandam; o MC insere-se entre A e B . Corrida limpa (1 ronda, 8 MCM).
02	colision_b	B e C mais próximos do <i>merge point</i>	B em conflito pede SLOWDOWN a C; C concede, mas no <i>recheck</i> B já não trava a tempo ⇒ várias rondas MERGE_GRANT(abort)/retry, até o MC se inserir atrás de B (entre B e C).
03	colision_all	C ainda mais próximo	B não consegue abrandar; depois C também não; rondas sucessivas de abort enquanto a folga não abre; o MC acaba por entrar no fim da via.
04	2mcs_standard	MC e A <i>ambos</i> na rampa	O MC (à frente na rampa) insere-se primeiro (cadeia B→C); só depois A (2.º MC) negocia, com conflito sucessivo com B e C, e entra no final .
05	2mcs_colision_b	2 MC; B, C mais próximos	MC em conflito com B (B não trava) ⇒ rondas de abort e cadeia na própria rampa (SLOWDOWN_REQUEST MC→A); MC entra entre B e C; depois A em conflito com C, que não trava, e A entra no fim.
06	2mcs_collision_all	2 MC; B, C muito próximos	MC em conflito com B e C (nenhum trava) mais cadeia na rampa MC→A; o MC entra a seguir a C e A entra a seguir ao MC .

Tabela 8: Os seis cenários da demonstração e o comportamento observado nas corridas.

A progressão é deliberada. De **01 a 03**, os veículos da via principal arrancam sucessivamente mais perto do *merge point*, passando de “B abranda a tempo” a “ninguém consegue travar e o MC espera que a folga se abra”. De **04 a 06** repete-se a mesma escalada, mas com *dois* Merge Cars na rampa, o que evidencia duas propriedades do protocolo: (i) os dois MC negociam de forma **independente e sequencial**, cada um corre a sua própria sessão; e (ii) a **cadeia de abrandamento aplica-se também na rampa** — SLOWDOWN_REQUEST MC→A (O6, Secção 4.4.1). Em todos os cenários “colisão”, as rondas de MERGE_CONFIRMED(abort) seguidas de retentativa são a salvaguarda **O5** em ação: o sistema nunca força a entrada, limitando-se a repetir a negociação até esta ser comprovadamente segura (ou a recorrer ao *stop-and-wait*). As evidências (*screenshots* e *logs* anotados) de cada cenário constam da Secção 6.

6 Resultados

[**TODO:** Por cenário: comportamento observado (conflito, cadeia de abrandamento, merge sem colisão) com evidência (screenshots da dashboard + logs anotados). Validação dos casos de colisão (sem protocolo colidiria; com protocolo é seguro). Caso de abort + retry. Caso fallback stop-and-wait. Multi-MC (cadeia também na rampa). Discussão: limitações e simplificações do modelo.]

7 Conclusão

[**TODO:** Síntese do alcançado vs objetivos; lições aprendidas; trabalho futuro.]

Referências